

SMALL LANGUAGE MODEL CHALLENGE

Parameter-Matched SwiGLU with Capacity Scaling and Dropout

Student: Li Zhoutong **Student ID:** 3036781272

Course: DASE7506 **Protocol:** 7506-mp1-wt2-v2 **Date:** 29 September 2026

1. Summary

The supplied 1.09M-parameter causal GPT reached 2.101260 full-test bits per byte (BPB). I replaced its GELU feed-forward sublayer with a parameter-matched SwiGLU gate, selected that mechanism on validation, and then trained a wider, deeper and regularised SwiGLU model for longer as the frozen final predictor. The controlled comparison improved validation BPB from 2.071083 to 2.010852 with essentially identical parameter count and training targets. The final model achieved 1.567424 validation BPB and 1.589531 CPU FP32 test BPB, a 24.35% relative test reduction from baseline.

Final test BPB	CPU score time	Peak RAM	Inference assets
1.589531	42.99 s (3.19x)	1.4972 GiB	20.46 MiB total

2. Task and baseline

The task is next-token prediction on the supplied WikiText-2 benchmark with a fixed training-fitted BPE-2048 tokenizer. Evaluation uses independent causal windows of 256 targets and reports summed negative log probability in bits divided by the raw UTF-8 byte count. Lower BPB is better. Development and model selection use validation; test is evaluated only after freezing the predictor.

The baseline is a four-block pre-LayerNorm decoder-only Transformer with width 128, four attention heads, learned absolute positions, tied token/output embeddings and a 4x GELU MLP. It has 1,088,256 parameters and is trained from random initialization for 1,200 updates with batch size 32: 9,830,400 processed targets.

I reproduced the baseline on Windows 11, Python 3.12.13 and PyTorch 2.7.1+cpu using four threads on an AMD Ryzen 7 4800HS. Training took 769.99 s; validation BPB was 2.071083 and test BPB was 2.101260. This matches the staff reference of approximately 2.10 BPB.

3. Method

3.1 Controlled mechanism: parameter-matched SwiGLU

The baseline MLP has one nonlinear feature path:

$$\text{GELU MLP}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2$$

I replace it with a gated path:

$$\text{SwiGLU}(x) = W_o [\text{SiLU}(W_g x + b_g) * (W_v x + b_v)] + b_o$$

The gate branch can suppress or pass each candidate value feature at every token position. Directly retaining hidden width 512 would add many parameters because SwiGLU has gate, value and output projections. I instead use hidden width 341: the student has 1,088,424 parameters, only 168 (0.015%) more than baseline. Attention, LayerNorm, positions, residual layout, initialization, optimizer, seed and sampling are held fixed. The original GELU model is therefore the removal ablation for the gate.

3.2 Final quality/cost candidate

After the controlled SwiGLU model improved validation, I froze the mechanism choice and allocated additional capacity and regularisation to the final candidate: width 256, eight heads, six blocks, SwiGLU hidden width 688 and dropout 0.1. It has 5,355,584 parameters and trains for 12,000 updates (98,304,000 targets). Training used a local GPU for speed, but the ranked result was measured on CPU FP32. This model intentionally changes width, depth, dropout and training duration, so it is a final quality/cost optimization rather than evidence that isolates SwiGLU.

4. Experimental design

Run	Width / heads	Steps	Targets	Parameters	Purpose
GELU baseline	128 / 4	1,200	9,830,400	1,088,256	Ablation / reference
Matched SwiGLU	128 / 4	1,200	9,830,400	1,088,424	Mechanism test
Final SwiGLU + dropout	256 / 8	12,000	98,304,000	5,355,584	Frozen score

All runs use seed 17, context 256, AdamW with the supplied schedule, gradient clipping and only the supplied training text. The fixed evaluator and tokenizer are unchanged. The 1,200-step pair is the required same-target comparison and key-mechanism ablation. The final model is selected solely because its 1.567424 validation BPB is lower than the earlier candidates.

5. Results

Method	Val BPB	Test BPB	Train time	CPU test	Checkpoint SHA-256
GELU baseline	2.071083	2.101260	769.99 s	13.46 s	7d869d...6d3f
Matched SwiGLU	2.010852	Not run	724.22 s	Not run	a555a3...3c94
Final SwiGLU + dropout	1.567424	1.589531	2,207.84 s GPU	42.99 s CPU	90163a...7802

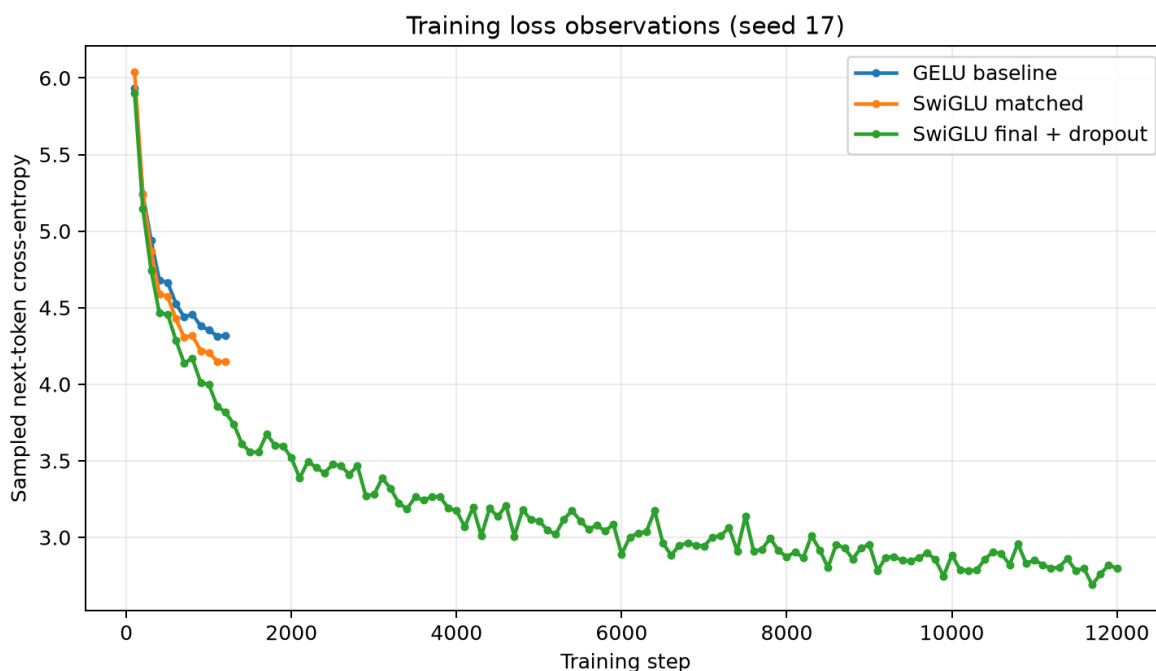


Figure 1. Logged training losses. The matched pair shares the same 1,200-step budget; the final dropout run continues to 12,000 updates.

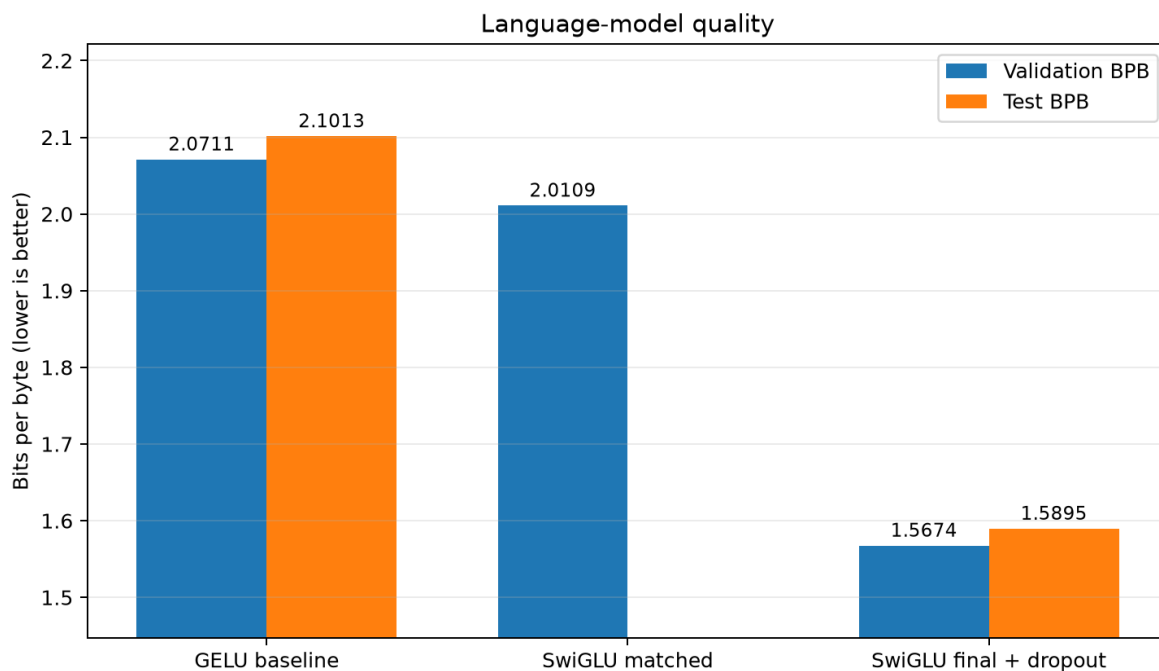


Figure 2. Validation and frozen test BPB. Lower is better; unselected models were not evaluated on test.

6. Analysis and trade-offs

6.1 What the controlled comparison establishes

At the same 9,830,400 targets, validation BPB decreases by 0.060232 (2.91% relative) when GELU is replaced by parameter-matched SwiGLU. The training-loss advantage becomes consistent after roughly 300 updates and persists through step 1,200. Because the models differ by only 168 parameters and all other listed conditions are held fixed, the paired evidence supports the hypothesis that multiplicative gating uses this feed-forward parameter budget more effectively.

The matched run took 724.22 s versus 769.99 s for baseline, but this single local timing should not be interpreted as an inherent SwiGLU speed advantage; background load and kernel variation were not controlled across repetitions.

6.2 Final score versus cost

The final model improves validation by 0.503660 BPB (24.32%) and test by 0.511730 BPB (24.35%) relative to baseline. It uses 4.92x parameters and 10x processed targets. CPU FP32 scoring took 42.99 s, or 3.19x baseline, below the 5x time limit. Peak process-tree working set was 1.4972 GiB. The checkpoint, student implementation and configuration total 20.46 MiB. These measurements remain below the 4 GiB RAM and 64 MiB asset limits.

Constraint	Limit	Measured	Status
CPU scoring	$\leq 5\times$ baseline	3.19x	Pass
Peak evaluation RAM	≤ 4 GiB	1.4972 GiB	Pass
Inference assets	≤ 64 MiB	20.46 MiB total	Pass

6.3 Limitations

The controlled evidence uses one seed, one small scale and one corpus, so it does not estimate run-to-run variance or prove that the same relative gate benefit generalizes. The final result cannot be attributed to SwiGLU alone because width and training duration also increase. The public test set was evaluated only after validation-based freezing, so no test-based hyperparameter sensitivity is available by design.

7. Reproducibility

From the code directory, install the pinned dependencies and run the contract tests before training:

```
python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cpu
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
```

Controlled SwiGLU training:

```
python train.py --implementation student --config configs/swiglu_128.json --device cpu --precision
fp32 --threads 4 --seed 17 --steps 1200 --batch-size 32 --run-dir runs/swiglu128-fair-s17
```

Final training and frozen evaluation:

```
python train.py --implementation student --config configs/swiglu_256x6_dropout.json --device cuda
--precision fp32 --threads 4 --seed 17 --steps 12000 --batch-size 32 --eval-every 2000 --run-dir
runs/swiglu256x6-dropout10-final12000-s17
python evaluate.py --checkpoint runs/swiglu256x6-dropout10-final12000-s17/checkpoint.pt --device
cpu --precision fp32 --threads 4 --split test
```

Final checkpoint SHA-256:

90163aed08c92f1affce1830907e63a62ef0b01ec32327c82a71088f8c747802. The supporting student_model.py and configuration are included in the repository and checkpoint bundle.

8. Conclusion

A parameter-matched SwiGLU gate provides a clear controlled validation improvement over the supplied GELU MLP. Scaling the selected design to width 256, six blocks and dropout 0.1, then training for 12,000 steps, produces 1.589531 CPU FP32 test BPB while remaining below the stated time and asset limits. The experiment separates evidence for the gate from the additional capacity/training trade-off and preserves the fixed data, tokenizer and evaluation protocol.

References

- [1] DASE7506 MP1 starter package and evaluation protocol, 2026.
- [2] Noam Shazeer. *GLU Variants Improve Transformer*. arXiv:2002.05202, 2020.
- [3] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. *Pointer Sentinel Mixture Models*. arXiv:1609.07843, 2016.